U-Boot `uEnv.txt` : Seri Porttan Yükleme ve Rootfs'i eMMC'ye Taşıma (BeagleBone Black)

Bu dokümanda dört şey yapacağız:

1. `uEnv.txt` dosyasının **satır satır ne yaptığını** anlayacağız.
2. Bu dosyayı SD karta kopyalamadan, **seri port (UART) üzerinden** doğrudan U-Boot'a göndereceğiz.
3. Challenge: Kernel ve DTB'yi **SD karttan** yükleyip rootfs'i **eMMC'den** bağlayacağız.
4. **ROOTFS** bölümünde hangi klasör ve dosyaların olduğunu ve bunlara **hangi komutlarla** ulaşacağımızı göreceğiz.

> **Not**
>
> Bu dokümandaki terminal görüntüleri, adımları göstermek için hazırlanmış **temsili** çıktılardır. Senin kartında U-Boot sürümü, dosya boyutları ve bölüm boyutları farklı görünebilir.
>
> SD karttan boot'un temelleri için: booting.md · eMMC'ye kurulum için: eMMC-booting.md

---

1. Temel Kavramlar

Kavram	Ne işe yarar?
`uEnv.txt`	U-Boot ortam değişkenlerini **metin olarak** tanımlayan dosya. Her satır `isim=değer` biçimindedir. U-Boot'un bu dosyayı açılışta otomatik okuyup okumaması, U-Boot'un yapılandırmasına bağlıdır (BBB'nin varsayılan U-Boot'u SD karttaki `uEnv.txt`'yi arar).
Ortam değişkeni (environment)	U-Boot'un ayarları ve makroları. `printenv` ile görülür, `setenv` ile değiştirilir, `saveenv` ile kalıcı kaydedilir.
Minicom	Bilgisayardan seri port (UART) üzerinden karta bağlanmayı sağlayan terminal programı. Kartın konsolu buraya düşer.
`loady`	U-Boot'ta **YMODEM** protokolüyle seri porttan dosya alıp RAM'e yazan komut.
`env import -t <adres> <boyut>`	RAM'deki `isim=değer` metnini okuyup U-Boot'un **aktif** ortamına ekler. `-t` "düz metin" demektir, `<boyut>` metnin bayt cinsinden uzunluğudur.

Bütün akış şöyle:

```
Bilgisayar (uEnv.txt) ──YMODEM / UART──► RAM 0x82000000 ──env import──► U-Boot ortamı ──boot──► Linux
```

---

2. `uEnv.txt` Satır Satır

Kullanacağımız dosya bu:

```
mypcip=setenv serverip 192.168.1.2
ipaddr=192.168.27.1
bootargs=console=ttyO0,115200 root=/dev/mmcblk0p2 rw
bootcmd=echo "**********Booting from memory**********";load mmc 0:1 0x82000000 uImage;load mmc 0:1 0x880
```

`mypcip` ve `ipaddr` — ağ ayarları

Satır	Anlamı
`mypcip=setenv serverip 192.168.1.2`	Bir **makro**. Kendi başına hiçbir şey yapmaz; `run mypcip` yazınca içindeki komut çalışır ve `serverip` (TFTP/NFS sunucusunun, yani bilgisayarın IP'si) `192.168.1.2` olur.
`ipaddr=192.168.27.1`	Kartın kendi statik IP adresi.

> **Uyarı**
>
> Bu örnekte kart `192.168.27.x`, sunucu `192.168.1.x` ağında. Ağ geçidi (`gatewayip`) tanımlanmadığı sürece TFTP/NFS çalışmaz. Ağdan boot yapacaksan iki adresi **aynı alt ağda** seç. SD karttan boot için bu iki satır kullanılmaz.

`bootargs` — kernel'e verilen parametreler

`bootargs=console=ttyO0,115200 root=/dev/mmcblk0p2 rw`

Parça	Anlamı
`console=ttyO0,115200`	Linux mesajları UART0'a (J1 header) 115200 baud ile yazılsın. `ttyO0` eski OMAP kernel'lerindeki addır; yeni kernel'lerde `ttyS0` olur.
`root=/dev/mmcblk0p2`	Root filesystem: MMC aygıtı **0**, bölüm **2** (`p2`).
`rw`	Rootfs okuma-yazma modunda bağlansın.

`bootcmd` — otomatik açılış komutu

`bootcmd`, `;` ile ayrılmış komutlardan oluşur. U-Boot bunları sırayla çalıştırır:

Sıra	Komut	Ne yapar?
1	`echo "****Booting from memory****"`	Ekrana mesaj basar (hangi yoldan açıldığını görmek için).
2	`load mmc 0:1 0x82000000 uImage`	`mmc 0:1` = SD kart, 1. bölüm (BOOT). Oradaki `uImage` dosyasını RAM'de `0x82000000` adresine yükler.
3	`load mmc 0:1 0x88000000 am335x-boneblack.dtb`	Aynı bölümdeki device tree dosyasını (DTB) `0x88000000` adresine yükler.
4	`bootm 0x82000000 - 0x88000000`	Kernel'i başlatır. Ortadaki `-` "initramfs yok" demektir, son adres DTB'nin adresidir.

```
RAM (DDR, 0x80000000'dan başlar)
├── 0x82000000  →  uImage   (kernel)
└── 0x88000000  →  am335x-boneblack.dtb
```

> **Not**
>
> `mmc 0` ↔ `/dev/mmcblk0` eşleşmesi çoğu kurulumda doğrudur ama **garanti değildir**. U-Boot tarafında `mmc list`, Linux tarafında `lsblk` ile kontrol et (bkz. Bölüm 4).

---

3. Uygulama: `uEnv.txt`'yi Seri Porttan Göndermek

Normalde `uEnv.txt` SD kartın BOOT bölümüne kopyalanır. Her değişiklikte SD kartı çıkar-tak yapmak yerine dosyayı **seri porttan** gönderip anında deneyebiliriz.

Adım 1 — Bilgisayarda dosyayı hazırla ve Minicom'u aç

Dosyanın içeriğini ve **boyutunu** kontrol et, sonra Minicom ile karta bağlan:

```
mehmet@ubuntu: ~/bbb
mehmet@ubuntu:~/bbb$ cat uEnv.txt
mypcip=setenv serverip 192.168.1.2
ipaddr=192.168.27.1
bootargs=console=ttyO0,115200 root=/dev/mmcblk0p2 rw
bootcmd=echo "**********Booting from memory**********";load mmc 0:1 0x82000000
uImage;load mmc 0:1 0x88000000 am335x-boneblack.dtb;bootm 0x82000000 - 0x88000000
mehmet@ubuntu:~/bbb$ ls -l uEnv.txt
-rw-rw-r-- 1 mehmet mehmet 269 Eki 16 17:05 uEnv.txt
mehmet@ubuntu:~/bbb$ sudo minicom -D /dev/ttyUSB0 -b 115200
```

> **İpucu**
>
> Windows'ta port adı `/dev/ttyUSB0` yerine `COM3` gibi olur. Minicom kurulumu için: minicom.md

Adım 2 — U-Boot'u durdur

Kartı aç. `Hit any key to stop autoboot` yazısı çıkınca **bir tuşa bas**. `U-Boot#` komut satırı gelmeli:

```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200
U-Boot SPL 2013.10 (Oct 16 2026 - 10:12:41)
reading u-boot.img

U-Boot 2013.10 (Oct 16 2026 - 10:12:41)

I2C:   ready
DRAM:  512 MiB
MMC:   OMAP SD/MMC: 0, OMAP SD/MMC: 1
Net:   cpsw, usb_ether
Hit any key to stop autoboot:  0    <-- herhangi bir tuşa bas
U-Boot#
```

Adım 3 — `loady` ile dosyayı RAM'e al

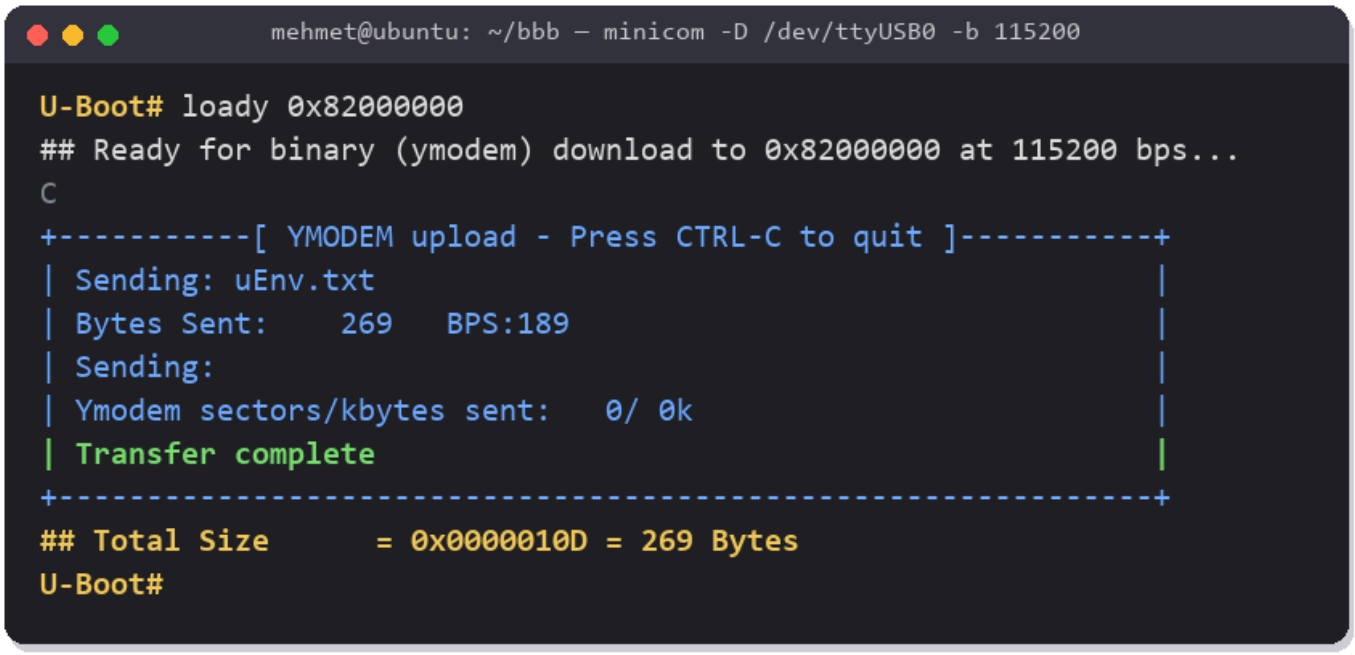
U-Boot'ta YMODEM alımını başlat:

```
U-Boot# loady 0x82000000
```

U-Boot `C` karakterleri basarak beklemeye başlar. Şimdi Minicom'da:

1. `Ctrl-A` → `S` (Send files)
2. Protokol olarak **ymodem** seç
3. `uEnv.txt` dosyasını seçip gönder

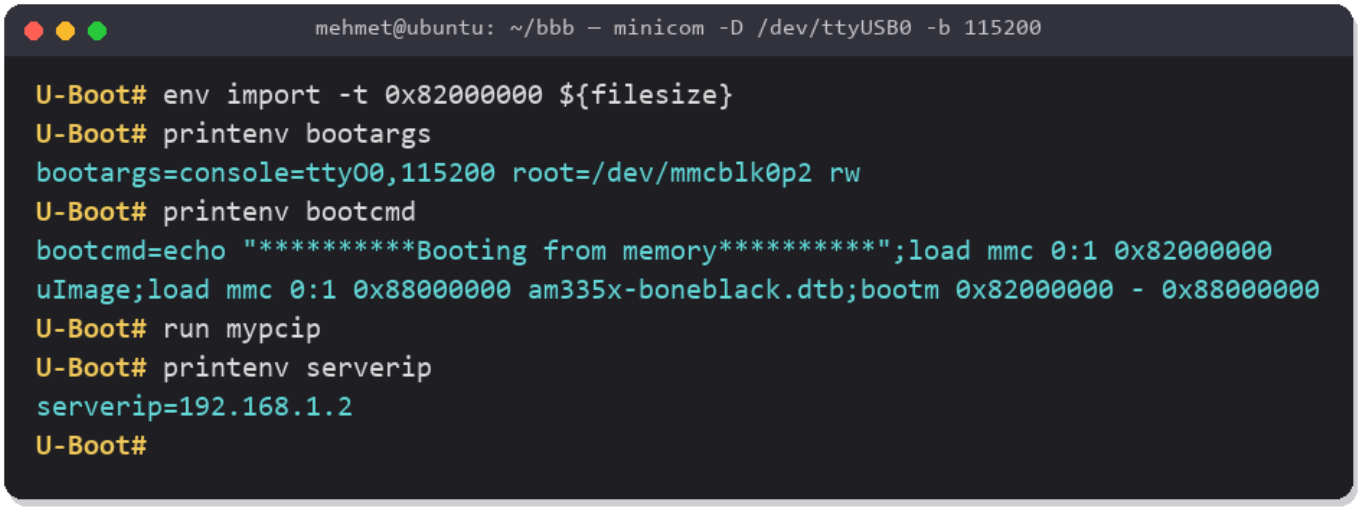
Transfer bitince U-Boot alınan dosyanın boyutunu yazar:



> **Not**
>
> `## Total Size = 0x0000010D = 269 Bytes` satırı önemli: Bu değer otomatik olarak **`filesize`** değişkenine yazılır. Bir sonraki adımda boyutu elle yazmak yerine `${filesize}` kullanabiliriz.

Adım 4 — `env import` ile ortama aktar ve kontrol et

```
U-Boot# env import -t 0x82000000 ${filesize}
U-Boot# printenv bootargs
U-Boot# printenv bootcmd
```



`printenv` çıktısı `uEnv.txt`'deki değerlerle aynıysa aktarım başarılıdır. `run mypcip` ile makronun da çalıştığını görebilirsin.

> **Uyarı**
>
> `${filesize}` yerine boyutu elle yazarsan **hex** olarak yaz (`0x10d` veya `10d`). U-Boot sayıları varsayılan olarak hex kabul eder; `269` yazarsan `0x269` = 617 bayt okur ve dosyanın sonundaki çöp veriyi de içe aktarmaya çalışır.

Adım 5 — Boot et

```
U-Boot# boot
```

`boot` komutu `bootcmd`'yi çalıştırır. Ekrana bizim `echo` mesajımız gelmeli:

```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200

U-Boot# boot
**********Booting from memory**********
reading uImage
4380760 bytes read in 245 ms (17.1 MiB/s)
reading am335x-boneblack.dtb
24884 bytes read in 9 ms (2.6 MiB/s)
## Booting kernel from Legacy Image at 82000000 ...
   Image Name:   Linux-3.8.13
   Image Type:   ARM Linux Kernel Image (uncompressed)
## Flattened Device Tree blob at 88000000
   Loading Device Tree to 8fff6000, end 8ffff133 ... OK

Starting kernel ...
```

Dikkat edilecekler

Konu	Açıklama
Adres çakışması	`uEnv.txt` `0x82000000`'a yüklendi; `bootcmd` de kernel'i aynı adrese yüklüyor. Sorun değil, çünkü `env import` dosyayı önce okuyup bitiriyor, kernel **sonra** üzerine yazılıyor. Sıra ters olursa kernel bozulur.
Geçicilik	`env import` sadece RAM'deki aktif ortamı değiştirir. Kart yeniden açılınca değişiklikler kaybolur.
Kalıcı yapmak	Her şeyin doğru olduğundan emin olduktan sonra `saveenv` ile kaydedilebilir. Bu komut kalıcı U-Boot ortamının üzerine yazar; geri dönmek için `env default -a` + `saveenv` gerekir.
Otomatik okumada `bootcmd`	BBB'nin varsayılan U-Boot'u SD karttaki `uEnv.txt`'yi kendi `bootcmd`'si **çalışırken** okur ve `uenvcmd` değişkenini arar. Bu yüzden SD karttaki `uEnv.txt`'de `bootcmd` tanımlamak o açılışı etkilemez; orada `uenvcmd` kullanılır (booting.md'deki örnek gibi). Seri porttan yüklemede ise `boot` komutunu biz verdiğimiz için `bootcmd` çalışır.

---

4. Challenge: Kernel ve DTB SD Karttan, Rootfs eMMC'den

Görev

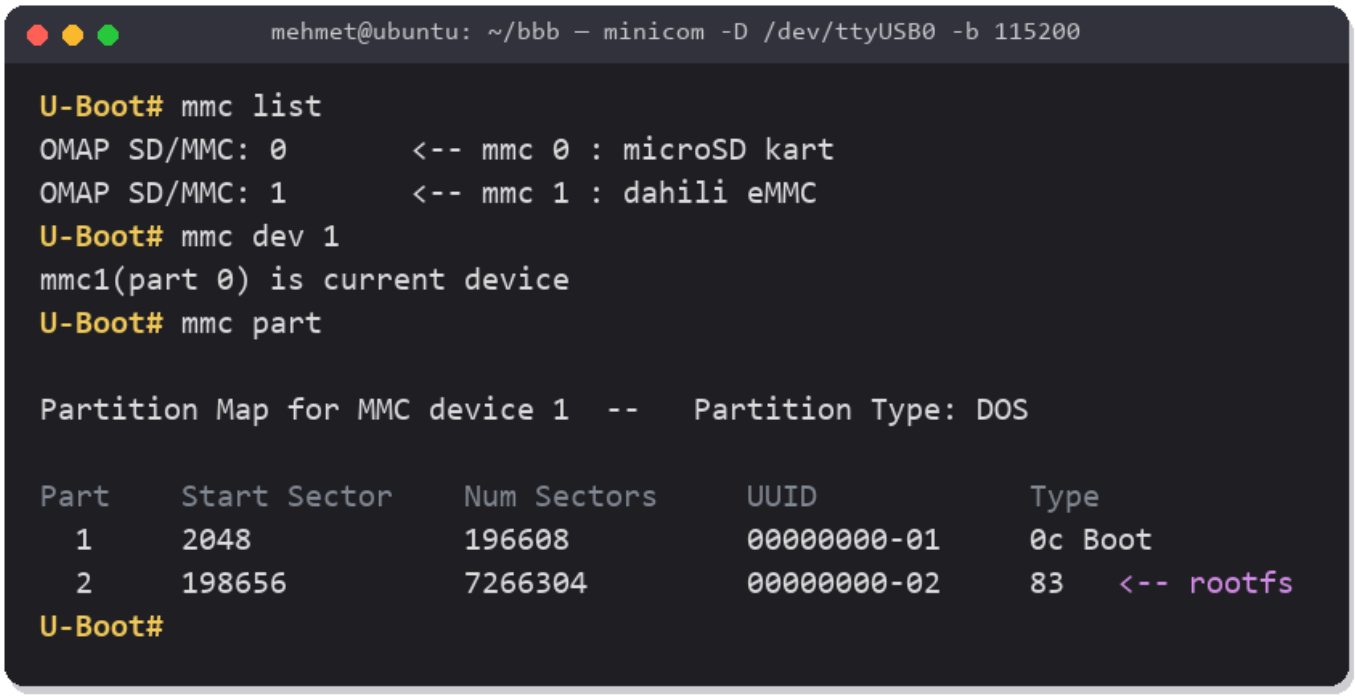
Ne?	Nereden?
Kernel (`uImage`)	**MMC0** — microSD kart, 1. bölüm
Device tree (`.dtb`)	**MMC0** — microSD kart, 1. bölüm
Root filesystem	**MMC1** — dahili eMMC, 2. bölüm

```
microSD (mmc 0)                      eMMC (mmc 1)
├── p1 BOOT   → uImage, .dtb  ──┐    ├── p1 BOOT
└── p2 ROOTFS                   │    └── p2 ROOTFS   ◄── root=/dev/mmcblk1p2
                                ▼
                    U-Boot RAM'e yükler → kernel başlar → rootfs'i eMMC'den bağlar
```

Adım 1 — Aygıt numaralarını doğrula

Önce varsayımımızı kontrol edelim: `mmc 0` gerçekten SD kart, `mmc 1` gerçekten eMMC mi ve eMMC'de 2. bölüm var mı?

```
U-Boot# mmc list
U-Boot# mmc dev 1
U-Boot# mmc part
```



eMMC'de `83` (Linux) tipinde 2. bölüm görünüyorsa rootfs için doğru yerdeyiz.

Adım 2 — Sadece `bootargs` 'ı değiştir

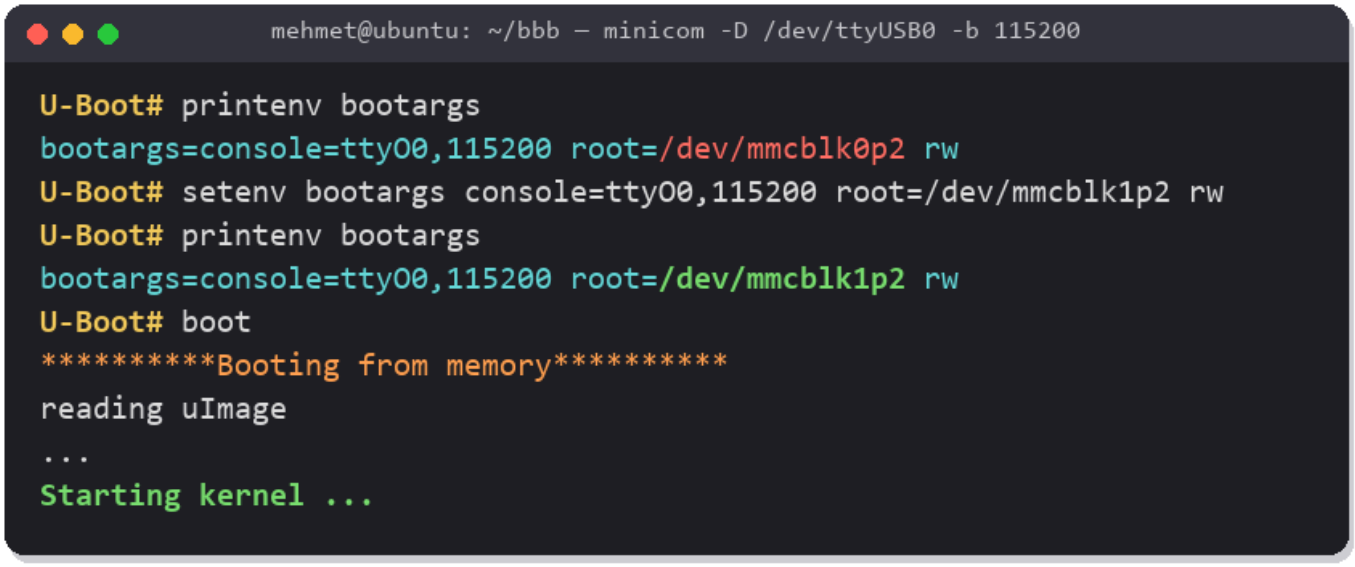
Kernel ve DTB hâlâ SD karttan yükleneceği için `bootcmd` **aynı kalır**. Değişen tek şey, kernel'e "rootfs nerede?" bilgisini veren `bootargs`:

	Değer
Eski — rootfs SD kartta	`bootargs=console=ttyO0,115200 root=/dev/mmcblk0p2 rw`
Yeni — rootfs eMMC'de	`bootargs=console=ttyO0,115200 root=/dev/mmcblk1p2 rw`

Bunu iki yolla yapabilirsin:

- **a)** `uEnv.txt`'deki satırı değiştirip Bölüm 3'teki gibi tekrar gönder.
- **b)** Doğrudan U-Boot'ta `setenv` ile değiştir:

```
U-Boot# setenv bootargs console=ttyO0,115200 root=/dev/mmcblk1p2 rw
U-Boot# boot
```



Adım 3 — Linux'ta sonucu doğrula

Kart açılınca rootfs'in gerçekten eMMC'den bağlandığını kontrol et:

```
cat /proc/cmdline   # kernel'e giden bootargs
lsblk               # hangi bölüm / olarak bağlı?
findmnt /           # rootfs'in kaynağı
```

```
root@beaglebone: ~ (seri konsol)
root@beaglebone:~# cat /proc/cmdline
console=ttyO0,115200 root=/dev/mmcblk1p2 rw
root@beaglebone:~# lsblk
NAME         MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
mmcblk0      179:0    0  7.4G  0 disk
|-mmcblk0p1  179:1    0   96M  0 part
`-mmcblk0p2  179:2    0  7.3G  0 part
mmcblk1      179:8    0  3.6G  0 disk
|-mmcblk1p1  179:9    0   96M  0 part
`-mmcblk1p2  179:10   0  3.5G  0 part /          <-- rootfs eMMC'de
root@beaglebone:~# findmnt /
TARGET SOURCE         FSTYPE OPTIONS
/      /dev/mmcblk1p2 ext4   rw,relatime
```

`/` bağlama noktası `mmcblk1p2`'nin yanında görünüyorsa challenge tamam.

Sonuç

Bileşen	Durum
Kernel ve DTB	microSD'nin 1. bölümünden RAM'e yüklenir.
Rootfs	Kernel açıldıktan sonra eMMC'nin 2. bölümünden bağlanır.
`bootcmd`	Değişmez; yükleme kaynağı hâlâ SD kart.
`bootargs`	`root=/dev/mmcblk1p2` olarak güncellenir.

> Önemli
>
> Bunun çalışması için: - eMMC'nin 2. bölümünde gerçekten bir Linux rootfs olmalı. - Kernel'de MMC sürücüsü ve rootfs'in dosya sistemi (ör. ext4) desteği **built-in** olmalı (veya bir initramfs'ten yüklenmeli). - Linux'taki numaralandırma U-Boot'unkiyle aynı olmalı. Değilse kernel `VFS: Unable to mount root fs` hatasıyla durur; bu durumda `root=/dev/mmcblk0p2` dene veya `root=PARTUUID=...` kullan.

---

5. ROOTFS Bölümünde Neler Var?

Root filesystem (rootfs), Linux'un `/` (kök) dizinidir. Kernel açıldıktan sonra `root=` ile gösterilen bölümü `/` olarak bağlar ve bundan sonra çalışan **her program, her ayar dosyası ve her kütüphane** buradan okunur.

Kernel tek başına bir işe yaramaz; kullanıcıya kabuk (shell), komutlar ve servisler rootfs ile gelir.

Rootfs'teki klasör düzeni rastgele değildir; **FHS (Filesystem Hierarchy Standard)** denen ortak bir standarda uyar. Bu yüzden Ubuntu'da, BeagleBone'daki Ångström'de veya eMMC'deki Debian'da aynı klasörleri görürüz.

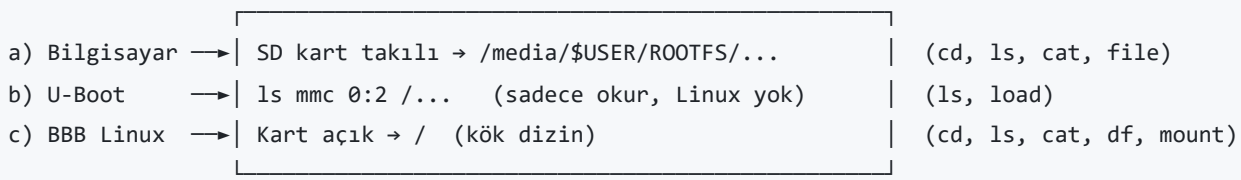
5.1 Klasörler ve Görevleri

Klasör	İçinde ne var?	BBB'de örnek	Bakmak için
`/bin`	Temel kullanıcı komutları (`ls` , `cp` , `cat` , `sh` ...)	`bash` , `busybox`	`ls /bin`
`/sbin`	Sistem yönetimi komutları (genelde root için)	`init` , `mount` , `ifconfig` , `reboot`	`ls /sbin`
`/boot`	Kernel ve device tree dosyaları	`uImage` , `am335x-boneblack.dtb`	`ls -l /boot`
`/etc`	Sistem **ayar** dosyaları (metin)	`hostname` , `fstab` , `passwd` , `os-release`	`cat /etc/hostname`
`/lib`	Paylaşılan kütüphaneler (`.so`) ve kernel modülleri	`libc.so.6` , `modules/3.8.13/`	`ls /lib/modules`
`/usr`	Ek programlar, kütüphaneler, belgeler (rootfs'in en büyük kısmı)	`/usr/bin` , `/usr/lib`	`du -sh /usr`
`/home`	Kullanıcıların kişisel klasörleri	`/home/root` (Ångström'de root'un ev dizini)	`ls /home`
`/root`	root kullanıcısının ev dizini (Debian/Ubuntu'da)	eMMC'deki Debian'da kullanılır	`ls /root`
`/var`	Zamanla değişen veriler: loglar, kuyruklar, önbellek	`/var/log`	`ls /var/log`
`/tmp`	Geçici dosyalar (açılışta temizlenir, genelde RAM'de)		`ls /tmp`
`/media` , `/mnt`	Harici diskleri (USB, SD) bağlama noktaları	`mount /dev/mmcblk1p1 /mnt`	`ls /mnt`
`/dev` 🔸	Donanım aygıt dosyaları	`mmcblk0p2` , `ttyO0`	`ls /dev`
`/proc` 🔸	Kernel ve süreç bilgileri	`cpuinfo` , `meminfo` , `cmdline`	`cat /proc/cpuinfo`
`/sys` 🔸	Aygıt ve sürücü ayarları (sysfs)	`/sys/class/leds/...`	`ls /sys/class`

Klasör	İçinde ne var?	BBB'de örnek	Bakmak için
`/run` 🔸	Çalışma anı bilgileri (PID dosyaları, soketler)		`ls /run`

> **Önemli**
>
> 🔸 işaretli klasörler **sanal (virtual)** dosya sistemleridir. Diskte gerçek dosya içermezler; kernel çalışırken içlerini **RAM'de** doldurur (`devtmpfs` , `proc` , `sysfs` , `tmpfs`). Bu yüzden SD kartı bilgisayara takıp baktığında bu klasörler **boş** görünür. Bu bir hata değildir.

5.2 ROOTFS'e Üç Yerden Ulaşabiliriz



Nereden?	Ne zaman?	Rootfs'in yolu
a) Bilgisayar	SD kart okuyucuya takılıyken, kartı açmadan dosya kopyalamak / düzeltmek için	`/media/$USER/ROOTFS`
b) U-Boot	Linux açılmadan önce, dosya gerçekten orada mı diye bakmak için	`mmc 0:2` (SD) veya `mmc 1:2` (eMMC)
c) BBB Linux	Kart açıldıktan sonra, seri konsol (Minicom) veya SSH üzerinden	`/`

5.3 a) Bilgisayardan Ulaşmak

SD kartı bilgisayara tak. Ubuntu bölümleri **etiket (label)** adlarıyla otomatik bağlar. Önce kartın nereye bağlandığını bul, sonra klasöre gir:

```
lsblk -o NAME,SIZE,FSTYPE,LABEL,MOUNTPOINT   # SD kart hangi aygıt, nereye bağlı?
cd /media/$USER/ROOTFS                         # rootfs'in içine gir
ls                                             # kök klasörleri listele
ls boot/                                       # kernel ve dtb burada
du -sh .                                       # rootfs'in toplam boyutu
```

```
mehmet@ubuntu: /media/mehmet/ROOTFS
mehmet@ubuntu:~$ lsblk -o NAME,SIZE,FSTYPE,LABEL,MOUNTPOINT /dev/sdb
NAME     SIZE FSTYPE LABEL  MOUNTPOINT
sdb      7.4G
├─sdb1     1G vfat   BOOT   /media/mehmet/BOOT
└─sdb2   6.3G ext3   ROOTFS /media/mehmet/ROOTFS   <-- buraya bağlı
mehmet@ubuntu:~$ cd /media/mehmet/ROOTFS
mehmet@ubuntu:/media/mehmet/ROOTFS$ ls
bin   dev  home  media  proc  sbin  tmp  var
boot  etc  lib   mnt    run   sys   usr
mehmet@ubuntu:/media/mehmet/ROOTFS$ ls boot/
am335x-boneblack.dtb  uImage  uImage-3.8.13
mehmet@ubuntu:/media/mehmet/ROOTFS$ du -sh .
412M    .
```

Burada iki şeyi gözlemle: sanal klasörler boş, programlar ise bilgisayarın değil **ARM** işlemcinin komut setiyle derlenmiş:

```
ls proc/ sys/ dev/      # boş görünür
file bin/bash           # "ARM" yazısını gör
cat etc/hostname        # BBB'nin hostname'i (başta / YOK)
```

```
mehmet@ubuntu: /media/mehmet/ROOTFS
mehmet@ubuntu:/media/mehmet/ROOTFS$ ls proc/ sys/ dev/
dev/:

proc/:

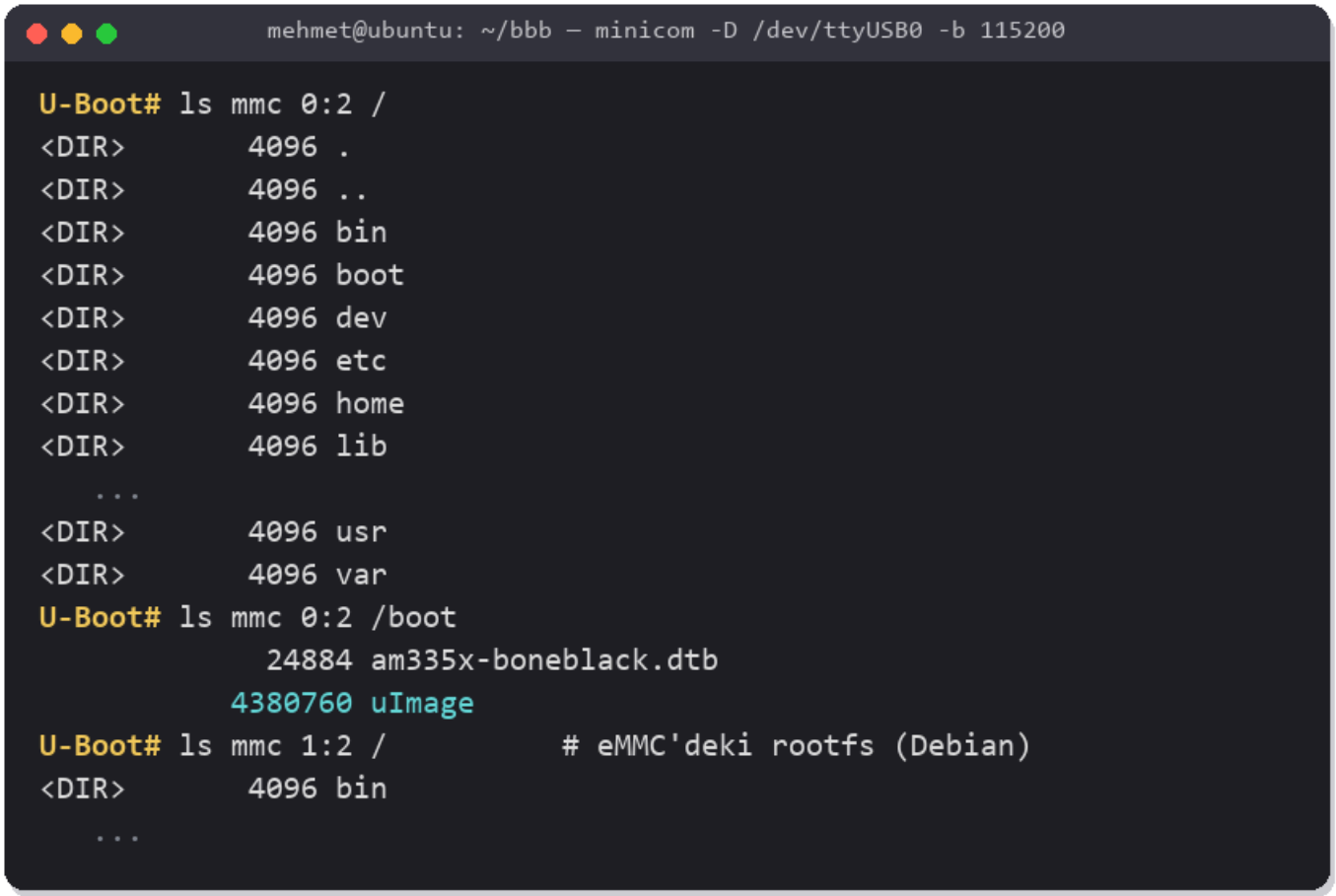
sys/:                  <-- boş! kernel çalışınca dolar
mehmet@ubuntu:/media/mehmet/ROOTFS$ file bin/bash
bin/bash: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV),
dynamically linked, interpreter /lib/ld-linux.so.3, for GNU/Linux 2.6.32
mehmet@ubuntu:/media/mehmet/ROOTFS$ cat etc/hostname
beaglebone
mehmet@ubuntu:/media/mehmet/ROOTFS$ cat /etc/hostname   <-- DİKKAT: baştaki /
ubuntu                     <-- bu bilgisayarın kendi /etc'si!
```

> **Uyarı**
>
> Bilgisayardayken yolun başına `/` **koyma**. `cat /etc/hostname` bilgisayarın **kendi** dosyasını okur; BBB'ninki `etc/hostname` (ROOTFS klasöründeyken) veya `/media/$USER/ROOTFS/etc/hostname` yolundadır. Bu karışıklıkla bilgisayarın `/etc/fstab` gibi dosyalarını yanlışlıkla bozmak mümkündür. Rootfs'e yazmak için `sudo` gerekir ve işin bitince kartı çıkarmadan önce `sync` + `umount` yap.
>
> `file bin/bash` çıktısında `ARM` yazdığı için bu programları bilgisayarda **çalıştıramazsın** (`./bin/bash` → `Exec format error`). Sadece okuyabilir/kopyalayabilirsin.

5.4 b) U-Boot'tan Ulaşmak

U-Boot dosya sistemini **okuyabilir** (ext2/3/4 ve FAT). Linux açılmadan önce kernel'in gerçekten orada olup olmadığını görmek için kullanışlıdır:

```
U-Boot# ls mmc 0:2 /          # SD kartın 2. bölümü (ROOTFS) kökü
U-Boot# ls mmc 0:2 /boot      # kernel ve dtb
U-Boot# ls mmc 1:2 /          # eMMC'nin 2. bölümü (Debian rootfs)
```



Komut	Anlamı
`ls mmc <aygıt>:<bölüm> <yol>`	Bölümdeki klasörü listeler

Komut	Anlamı
`load mmc 0:2 0x82000000 /boot/uImage`	Dosyayı RAM'e yükler (bkz. booting.md)
`ext4ls` / `ext2ls` / `fatls`	Eski U-Boot sürümlerinde dosya sistemine özel `ls` komutları

> **Not**
>
> U-Boot'ta `cd` yoktur ve dosya içeriğini `cat` ile göremezsin; her komutta tam yolu yazarsın. U-Boot rootfs'e normalde **yazmaz**, sadece okur.

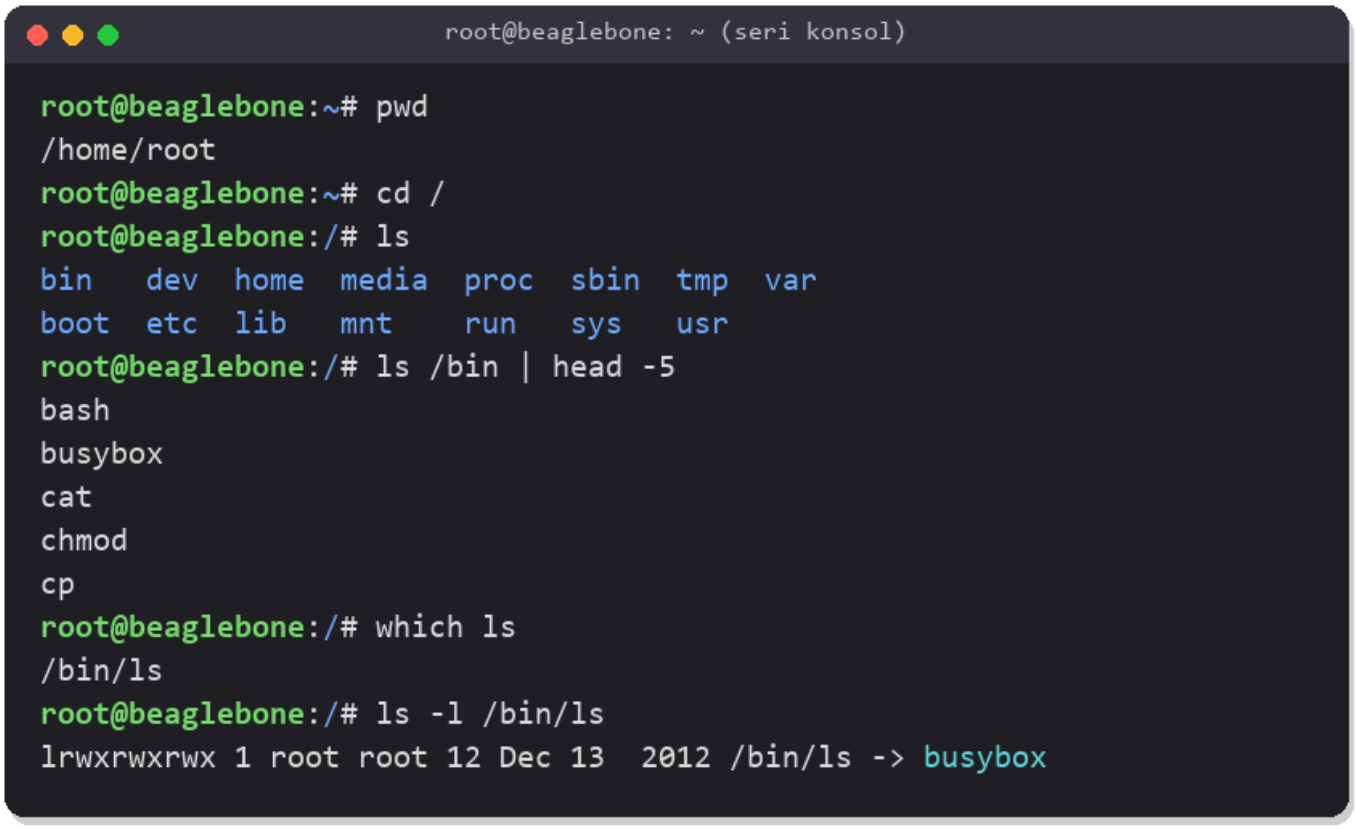
5.5 c) BBB Linux'tan Ulaşmak

Kart açılıp `root` olarak giriş yaptıktan sonra rootfs artık `/` dizinidir. Dolaşmak için temel komutlar:

Komut	Görevi
`pwd`	Şu an hangi klasördeyim?
`cd /` · `cd /etc` · `cd ..` · `cd ~`	Kök dizine / bir klasöre / bir üste / ev dizinine git
`ls` · `ls -l` · `ls -a`	Listele / ayrıntılı (izin, boyut, tarih) / gizli dosyalarla
`cat <dosya>` · `head -n 5` · `less`	Metin dosyasını göster / ilk satırları / sayfa sayfa
`which <komut>`	Bir komutun hangi dosya olduğunu bul
`find / -name "*.dtb" 2>/dev/null`	Rootfs'te dosya ara
`df -h /` · `du -sh <klasör>`	Rootfs'in doluluk oranı / klasör boyutu
`mount`	Hangi bölüm nereye bağlı?

Kök dizin ve `/bin`

```
pwd                # /home/root  (Ångström'de root'un ev dizini)
cd /
ls
ls /bin | head -5
which ls
ls -l /bin/ls      # Ångström'de birçok komut busybox'a link
```



> İpucu
>
> **BusyBox**, `ls`, `cp`, `mount` gibi yüzlerce komutu **tek bir küçük programda** toplayan araçtır. Gömülü sistemlerde yer kazanmak için yaygın kullanılır; `/bin/ls` gibi dosyalar aslında `busybox`'a giden **sembolik linklerdir**.

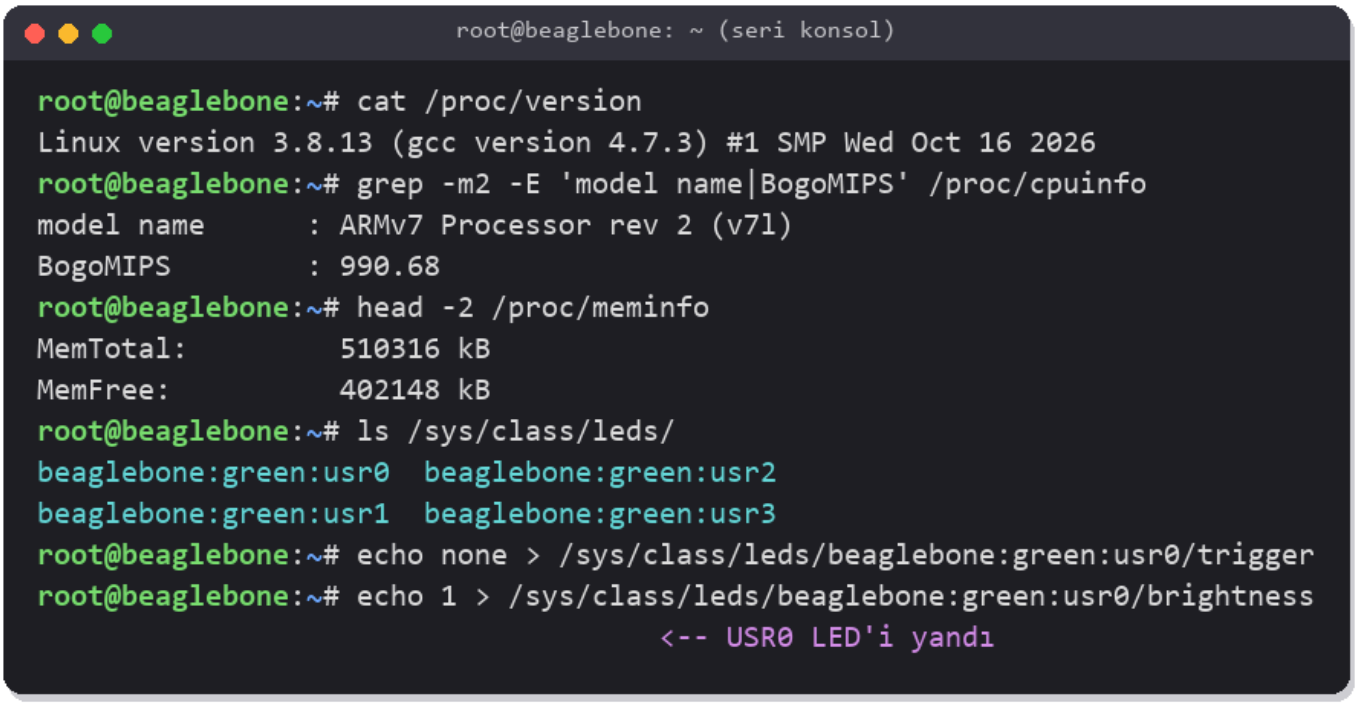
`/etc` — ayar dosyaları

```
cat /etc/hostname      # kartın adı
cat /etc/os-release    # hangi dağıtım, hangi sürüm
cat /etc/fstab         # açılışta hangi bölüm nereye bağlanacak
head -2 /etc/passwd    # kullanıcı listesi
```

```
root@beaglebone: ~ (seri konsol)
root@beaglebone:~# cat /etc/hostname
beaglebone
root@beaglebone:~# cat /etc/os-release
NAME="Angstrom"
VERSION="v2012.12"
ID="angstrom"
root@beaglebone:~# cat /etc/fstab
# <file system>  <mount pt>  <type>  <options>          <dump> <pass>
rootfs           /           auto    defaults            1      1
proc             /proc       proc    defaults            0      0
tmpfs            /tmp        tmpfs   defaults            0      0
root@beaglebone:~# head -2 /etc/passwd
root::0:0:root:/home/root:/bin/sh
daemon:*:1:1:daemon:/usr/sbin:/bin/sh
```

`/proc` ve `/sys` — kernel ile konuşmak

```
cat /proc/version                                  # kernel sürümü
cat /proc/cmdline                                  # U-Boot'un verdiği bootargs
grep -m2 -E 'model name|BogoMIPS' /proc/cpuinfo    # işlemci bilgisi
head -2 /proc/meminfo                              # RAM bilgisi
ls /sys/class/leds/                                # kartın USR LED'leri
echo none > /sys/class/leds/beaglebone:green:usr0/trigger
echo 1    > /sys/class/leds/beaglebone:green:usr0/brightness   # LED'i yak (0 = söndür)
```



`/sys` altındaki dosyalara yazmak, sürücüye doğrudan komut vermek demektir. `trigger` dosyasını `none` yapmadan `brightness`'a yazarsan LED'in varsayılan görevi (ör. `heartbeat`) senin değerini hemen ezer.

`/dev`, `/boot` ve disk kullanımı

```
ls /dev/mmcblk* /dev/ttyO*   # MMC bölümleri ve seri portlar
ls -l /boot                  # kernel ve dtb
df -h /                      # rootfs ne kadar dolu, hangi bölümden geliyor
du -sh /usr /lib /etc        # en çok yer kaplayan klasörler
mount | grep -E ' / | /proc | /sys | /dev '   # gerçek ve sanal bağlamalar
```

```
root@beaglebone: ~ (seri konsol)
root@beaglebone:~# ls /dev/mmcblk* /dev/ttyO*
/dev/mmcblk0    /dev/mmcblk0p2  /dev/mmcblk1p1  /dev/ttyO0
/dev/mmcblk0p1  /dev/mmcblk1    /dev/mmcblk1p2
root@beaglebone:~# ls -l /boot
-rw-r--r-- 1 root root   24884 Dec 13  2012 am335x-boneblack.dtb
lrwxrwxrwx 1 root root      14 Dec 13  2012 uImage -> uImage-3.8.13
-rw-r--r-- 1 root root 4380760 Dec 13  2012 uImage-3.8.13
root@beaglebone:~# df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/mmcblk0p2  6.2G  412M  5.5G   7% /
root@beaglebone:~# du -sh /usr /lib /etc 2>/dev/null
287M    /usr
61M     /lib
3.1M    /etc
root@beaglebone:~# mount | grep -E ' / | /proc | /sys | /dev '
/dev/mmcblk0p2 on / type ext3 (rw,relatime)
devtmpfs on /dev type devtmpfs (rw,relatime)
proc on /proc type proc (rw,relatime)
sysfs on /sys type sysfs (rw,relatime)
```

`mount` çıktısında `/` gerçek bir diskten (`/dev/mmcblk0p2` , `ext3`) gelirken `/dev` , `/proc` ve `/sys` 'in tipi `devtmpfs` , `proc` , `sysfs` olarak görünür. Bu, 5.1'deki sanal klasörlerin kanıtıdır. Rootfs'i eMMC'den açtıysan (Bölüm 4) `df -h /` satırında `/dev/mmcblk1p2` görürsün.

---

6. Hızlı Tekrar

Komut / Değişken	Görevi
`uEnv.txt`	U-Boot ortam değişkenlerini metin olarak tanımlar
`loady <adres>`	YMODEM ile seri porttan RAM'e dosya alır, boyutu `filesize` 'a yazar
`env import -t <adres> ${filesize}`	RAM'deki metin değişkenlerini aktif ortama aktarır
`printenv <isim>`	Değişkenin değerini gösterir
`setenv <isim> <değer>`	Değişkeni değiştirir (geçici)

Komut / Değişken	Görevi
`run <makro>`	Değişkendeki komutları çalıştırır (ör. `run mypcip`)
`boot`	`bootcmd`'yi çalıştırır
`bootcmd`	Otomatik açılışta çalışan komut dizisi
`bootargs`	Linux kernel'ine verilen açılış parametreleri
`root=/dev/mmcblk1p2`	Rootfs'in eMMC'nin 2. bölümünde olduğunu söyler
`mmc list` / `mmc part`	U-Boot'un gördüğü MMC aygıtlarını ve bölümleri listeler
`saveenv`	Aktif ortamı kalıcı olarak kaydeder (dikkatli kullan)
`/media/$USER/ROOTFS`	SD kart bilgisayara takılıyken rootfs'in yolu
`ls mmc 0:2 /boot`	U-Boot'tan rootfs'teki klasörü listeler
`/bin`, `/sbin`, `/usr`	Programlar
`/etc`	Ayar dosyaları (`hostname`, `fstab`, `passwd`)
`/boot`	Kernel (`uImage`) ve device tree (`.dtb`)
`/lib`	Kütüphaneler ve kernel modülleri
`/dev`, `/proc`, `/sys`	Sanal klasörler; kernel çalışırken dolar
`cat /proc/cmdline`	Kernel'e giden bootargs
`df -h /` · `mount`	Rootfs hangi bölümden geliyor?